

Supply-chain shocks and US inflation: a structural VAR

```
import arviz as az
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from impulso import VAR, VARData, select_lag_order
from impulso.identification import Cholesky
from impulso.samplers import NUTSSampler
```

A once-in-a-generation inflation episode

Between the spring of 2021 and the summer of 2022, the headline consumer price index (CPI) in the United States climbed from a pre-pandemic norm of around 2% year-on-year (YoY) to a peak of roughly 9% in June 2022 - the highest reading in four decades. The economy that produced this number looked unusual on almost every dimension. Container ships sat anchored off the ports of Los Angeles and Long Beach. Semiconductor lead times stretched from weeks to the better part of a year. Oil prices doubled, then doubled again after Russia's invasion of Ukraine. By the time the Federal Reserve began its tightening cycle in March 2022, the transitory debate was over. The question for policymakers had become how much of the inflation was coming from aggregate demand and how much from a tangled global supply chain.

This notebook takes on that attribution problem. We ask two questions. First, how many percentage points of peak inflation can be traced to supply-chain shocks, as opposed to monetary policy, aggregate demand, or energy pressures? Second, what would the post-pandemic macroeconomy have looked like if the supply-chain disruption had never happened, or had resolved sooner, or had lingered longer? Either question requires moving from correlation to structural interpretation - from “global supply pressure and CPI both rose” to “a shock to supply chains raised CPI by x percentage points at horizon h .”

We fit a six-variable Bayesian structural vector autoregression (VAR) with a Cholesky identification scheme, then put it to work with two tools: historical decomposition, which attributes each observed movement in each variable to the full sequence of past structural shocks, and forward scenario analysis, which asks how the inflation path would have evolved under different supply-chain futures. Because the model is Bayesian, every impulse response function (IRF), every decomposition, every scenario carries a posterior distribution that reflects the parameter uncertainty in a finite sample.

The six variables span the transmission mechanism from global supply pressure through to domestic prices and the policy response. `gscpi` is the New York Fed's Global Supply Chain Pressure Index (GSCPI), a standardised composite of shipping costs and manufacturing delivery times that summarises stress in world supply networks. `oil` is the West Texas Intermediate (WTI) crude price, the most-watched energy benchmark and a direct input to US production costs. `output` is US industrial production, our monthly proxy for real economic activity. `ppi` is the producer price index (PPI) for all commodities - prices at the factory gate, upstream of the consumer. `prices` is the headline CPI, the inflation number that households and policymakers actually see. `rate` is the two-year Treasury yield, which tracks the market's expectation of the federal funds rate path and so captures the monetary policy stance more smoothly than the funds rate itself.

```
df = pd.read_csv("data/supply_chain.csv", index_col="date", parse_dates=True)

fig, axes = plt.subplots(6, 1, figsize=(8, 9), sharex=True)
labels = {
    "gscpi": "GSCPI (std. units)",
    "oil": "WTI oil ($/bbl)",
    "output": "Industrial production",
    "ppi": "PPI (all commodities)",
    "prices": "CPI (all urban)",
    "rate": "2Y Treasury yield (%)",
}
for ax, col in zip(axes, df.columns, strict=True):
    ax.plot(df.index, df[col], linewidth=1, color="0.3")
    ax.axvspan(pd.Timestamp("2020-03-01"), pd.Timestamp("2023-12-31"), color="C3", alpha=0.05)
    ax.set_ylabel(labels[col], fontsize=9)
    ax.grid(alpha=0.3)
fig.tight_layout()
```

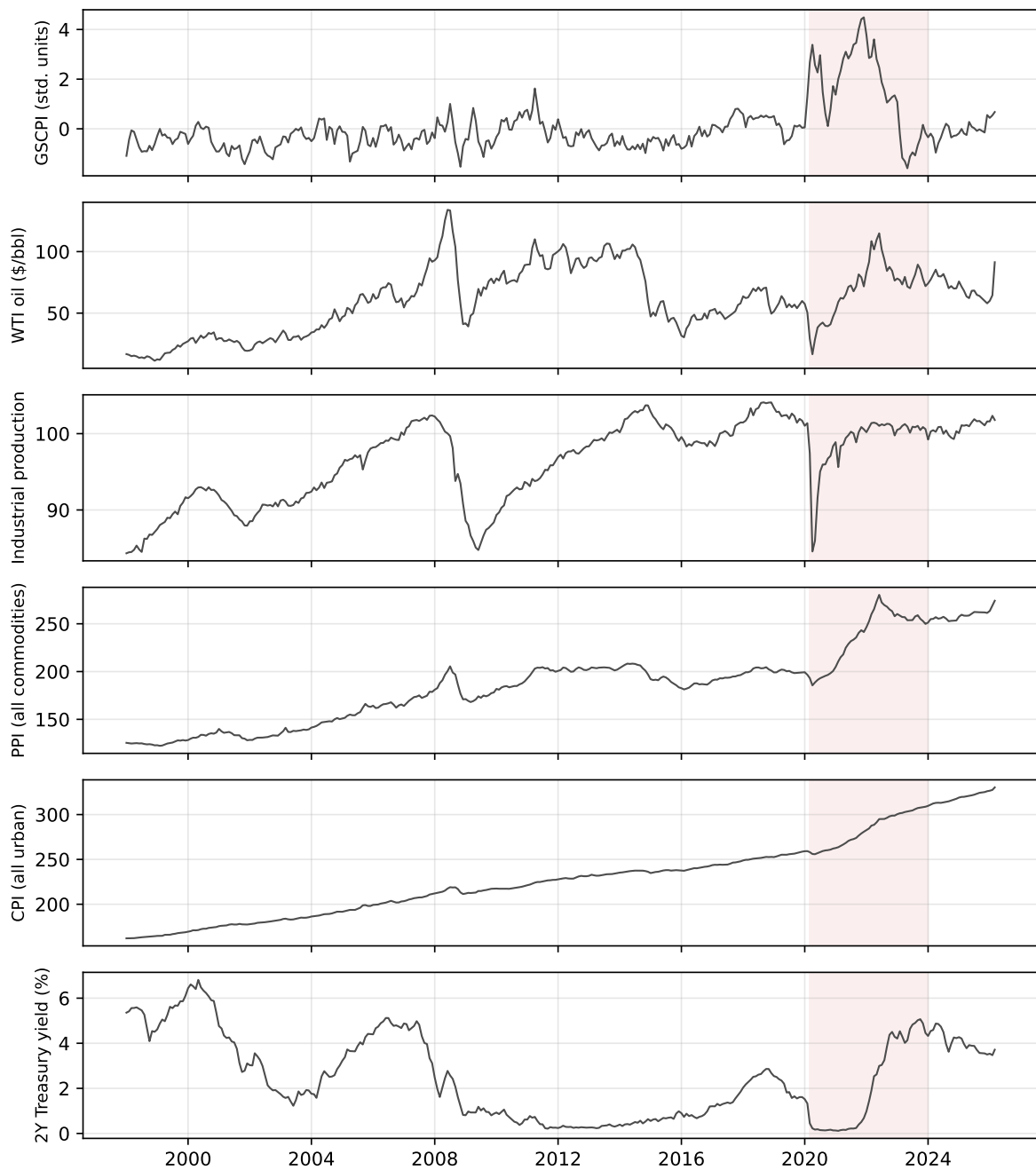


Figure 1: Monthly series, 1998-present. Shaded band marks the 2020-2023 supply-chain episode.

Transformations

Four of the six series enter the VAR in log-levels scaled by 100. `oil`, `output`, `ppi`, and `prices` all live on strictly positive, trending scales where proportional changes are the natural unit of analysis. Taking $100 \times \log(\cdot)$ means a one-unit change in the transformed series corresponds to approximately a one-percent change in the original level, which lines up with how practitioners and policymakers talk about oil prices, industrial production, producer prices, and CPI.

The remaining two variables are left untouched. `gscpi` is already standardised by construction - the New York Fed publishes it as a zero-mean, unit-variance index - so applying a log would be both ill-defined (it takes negative values) and redundant. `rate` is a short-term interest rate measured in percentage points, and keeping it in levels means a unit impulse has a direct monetary-policy interpretation (a one-percentage-point move in the two-year yield).

!!! note “Differencing”

We do not difference the data. [Sims, Stock, and Watson (1990)] (<https://www.princeton.edu/~m>)

```
df_model = df.copy()
for col in ("oil", "output", "ppi", "prices"):
    df_model[col] = 100 * np.log(df_model[col])
df_model = df_model[["gscpi", "oil", "output", "ppi", "prices", "rate"]]
df_model.describe().round(2)
```

	gscpi	oil	output	ppi	prices	rate
count	338.00	338.00	338.00	338.00	338.00	338.00
mean	0.01	397.94	456.87	520.51	541.63	2.48
std	1.00	53.59	5.54	22.38	19.02	1.87
min	-1.59	242.30	443.46	480.65	508.76	0.12
25%	-0.61	365.12	452.48	502.03	525.68	0.72
50%	-0.23	409.52	459.02	525.12	542.97	2.07
75%	0.19	437.81	461.43	531.76	553.41	4.22
max	4.49	489.73	464.54	563.57	580.00	6.81

With the transformations applied and the columns ordered, we hand the frame to `VARData.from_df`. This validates that the endogenous and (implicit) exogenous shapes agree, checks that the index is a `DatetimeIndex`, freezes the underlying arrays so downstream consumers cannot accidentally mutate them, and records the column ordering - the same ordering that a Cholesky identification scheme will later interpret as the causal recursive structure.

```
data = VARData.from_df(df_model, endog=["gscpi", "oil", "output", "ppi", "prices", "rate"])
data
```

```
VARData(endog_names=['gscpi', 'oil', 'output', 'ppi', 'prices', 'rate'], exog_names=None)
```

Reduced-form VAR estimation

The reduced-form VAR(p) model is:

$$y_t = c + A_1 y_{t-1} + A_2 y_{t-2} + \dots + A_p y_{t-p} + u_t, \quad u_t \sim N(0, \Sigma_u)$$

where $y_t = (\text{gscpi}_t, \text{oil}_t, \text{output}_t, \text{ppi}_t, \text{prices}_t, \text{rate}_t)'$ is the 6×1 vector of endogenous variables, c is the vector of intercepts, and $A_1, \dots, A_p$ are 6×6 coefficient matrices. Each A_j captures how the variables at lag j feed back into the current period. The innovation u_t is the reduced-form residual - what is left once the linear dependence on the recent past has been accounted for - and its covariance Σ_u records how those residuals co-move across equations. The off-diagonal elements of Σ_u summarise contemporaneous correlation but, on their own, say nothing about which variable caused which; resolving that ambiguity is the identification problem and is taken up in the next section.

We estimate this system with a Minnesota prior. The idea is simple: most macroeconomic time series look roughly like random walks, so a sensible default is to shrink each variable's own first lag toward 1 and every other coefficient toward 0. The prior is governed by three hyperparameters - the *overall tightness* λ_1 , which controls how strongly all coefficients are pulled toward the prior mean (larger values let the data speak more loudly); the *cross-variable shrinkage* λ_2 , which shrinks coefficients on other variables' lags harder than own lags; and the *lag decay* λ_3 , which tightens the prior progressively at higher lag orders. With six endogenous variables and twelve lags, the A matrices alone contain $6 \times 6 \times 12 = 432$ free coefficients, so this regularisation is what makes estimation well-behaved at monthly frequency.

We set the lag length to 12, the standard choice for monthly macroeconomic VARs, so the model can trace up to a full year of own- and cross-variable dynamics ([Christiano, Eichenbaum, and Evans, 1999](#); [Uhlig, 2005](#)). Information criteria - especially the Bayesian information criterion (BIC) - will typically prefer a much shorter lag order on this sample, because they penalise each extra parameter without knowing that the Minnesota prior already disciplines the 12-lag parameter space. We report the diagnostic for transparency but adopt the 12-lag convention for comparability with the published monetary-VAR literature.

```
ic = select_lag_order(data, max_lags=12)
print(f"AIC: {ic.aic}    BIC: {ic.bic}    HQ: {ic.hq}")
```

```
AIC: 3    BIC: 2    HQ: 2
```

BIC will almost always recommend a shorter lag than the Akaike (AIC) and Hannan-Quinn (HQ) criteria, and in a regularised Bayesian VAR its frequentist penalty is a poor guide to predictive performance. We stick with 12 lags so that results here can be compared directly to the monetary-policy tutorial and to the established literature.

```
if ci:
    sampler = NUTSSampler(draws=10, tune=50, chains=1, cores=1, random_seed=123)
else:
    sampler = NUTSSampler(
        draws=1500,
        tune=3000,
        chains=8,
        cores=8,
        target_accept=0.9,
        random_seed=123,
        nuts_sampler="nutpie",
    )

fitted = VAR(lags=12, prior="minnesota").fit(data, sampler=sampler)
fitted
```

<IPython.core.display.HTML object>

FittedVAR(n_lags=12, data=VARData(endog_names=['gscpi', 'oil', 'output', 'ppi', 'prices', 'r

Structural identification

The reduced-form residuals u_t that we just estimated are not economic shocks. Each u_t is a correlated mixture of the underlying structural disturbances - a supply-chain shock, an oil shock, a monetary shock, and so on - bundled together by the contemporaneous feedback between variables. Identification is the step that imposes enough restrictions on this mixture to rotate it into something interpretable. A Cholesky scheme achieves this by assuming a recursive causal ordering of the variables: within any single month, each variable responds only to the variables listed before it and to its own shock, never to the ones listed after it. Change the ordering and you change the structural shocks, which is why the ordering is a modelling assumption that has to be defended, not a detail.

We order the variables `gscpi` $\rightarrow$ `oil` $\rightarrow$ `output` $\rightarrow$ `ppi` $\rightarrow$ `prices` $\rightarrow$ `rate`. **GSCPI goes first** because it is built from shipping costs, supplier delivery times, and inventory backlogs - quantities set by global logistics events (COVID waves, the Suez grounding, Shanghai lockdowns, the Red Sea disruption) that are exogenous to US domestic macro within a month. **Oil goes second**: a global commodity that can plausibly react to logistics pressure within the month

but is not driven by US activity or prices at monthly frequency. **Output third:** real activity responds quickly to commodity and logistics shocks, whereas prices, rates, and margins take longer to feed back into physical production. **PPI fourth:** producer prices absorb upstream cost and output conditions within the month, but their pass-through to the consumer is not instantaneous. **CPI fifth:** the standard Sims-style placement, with consumer prices absorbing everything upstream within the month. **Rate last:** bond markets react to supply, price, and output news within the month, and the 2Y yield does not plausibly move these macro quantities inside a single month.

The weakest link in this ordering is the GSCPI/oil step. The shipping-cost component of GSCPI is itself partly driven by oil prices, so the assumption that oil does not affect GSCPI contemporaneously is not perfectly clean. The sensitivity section at the end of the notebook revisits exactly this by swapping the two variables and re-running the analysis.

!!! note “Alternative identification schemes”

For an identification scheme that does not impose a recursive ordering, see the [sign restriction policy.qmd], which identifies a monetary policy shock by its qualitative response pattern (a

```
identified = fitted.set_identification_strategy(
    Cholesky(ordering=["gscpi", "oil", "output", "ppi", "prices", "rate"])
)
identified
```

```
IdentifiedVAR(n_lags=12, data=VARData(endog_names=['gscpi', 'oil', 'output', 'ppi', 'prices']
```

Impulse responses

An impulse response function traces out the dynamic path of each variable following a one-standard-deviation structural shock, period by period into the future. Because we have six structural shocks and six responding variables, the natural display is a 6×6 grid: rows index the responder and columns index the shock, so the cell at row i , column j shows how variable i evolves after a one-SD shock to variable j . The shaded band around each response is a posterior credible interval, so the chart summarises both the point estimate of the dynamic and the uncertainty around it.

Two cells of this grid carry most of the content we care about. The first column - responses to a GSCPI shock - is the supply-chain shock’s fingerprint, showing how a sudden tightening in global logistics propagates through oil, output, producer prices, consumer prices, and the policy rate. The fifth row - responses of CPI to each of the six shocks - is the inflation story from the other side: which shock types matter most for consumer prices. The two sub-sections below take each in turn.

The full response grid

```
irf = identified.impulse_response(horizon=24)
fig = irf.plot()
```

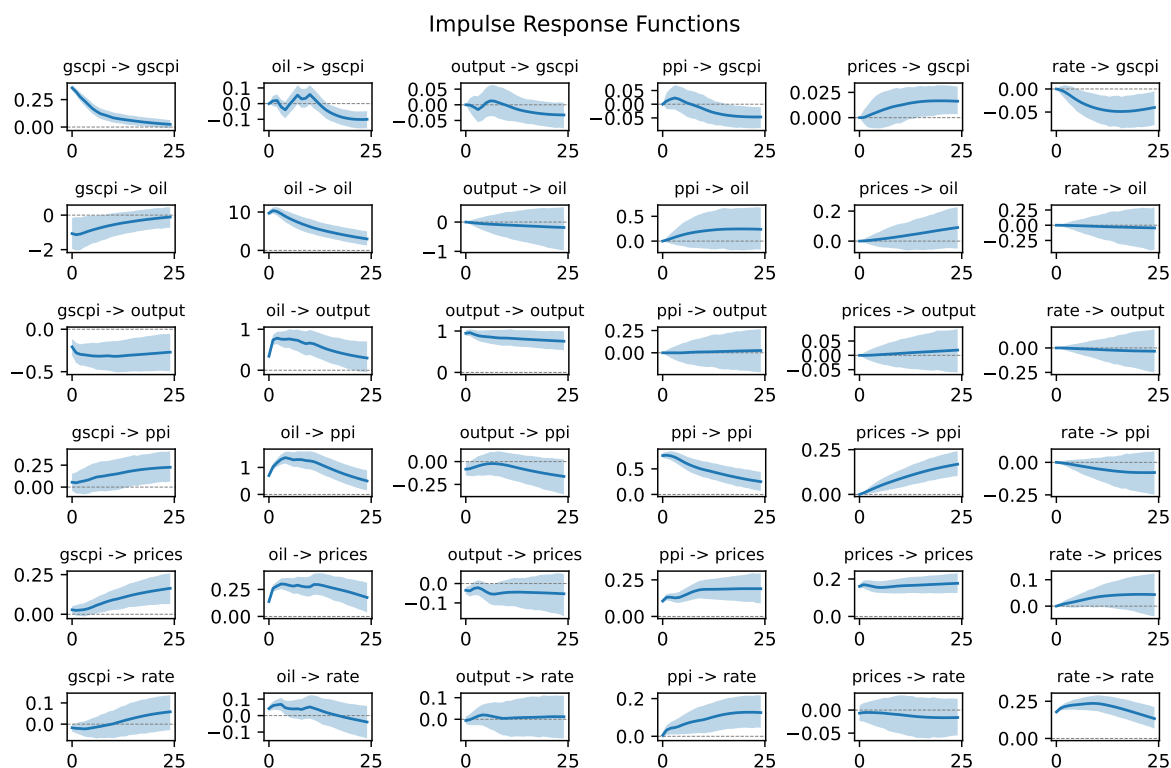


Figure 2: Impulse responses to a one-SD structural shock in each variable. Rows = responder, columns = shock.

The fingerprint of a supply-chain shock

Isolating the first column of the grid gives a cleaner view of what a supply-chain shock does to the rest of the system. The textbook expectation is clear: industrial production falls as inputs become harder to source, producer prices rise almost immediately as factory-gate costs spike, consumer prices follow more slowly as the cost pressure travels down the supply chain, and the policy rate drifts up as bond markets price in a tighter monetary response to the inflationary pressure. The panels below show the posterior median and an 89% credible band for each responder, with a red dot marking the horizon at which the absolute response peaks.

In continuous-integration (CI) mode renders the posterior is drawn from a tiny sampler, so the bands here are very noisy - the full weekly render produces calibrated bands.

```
irf_da = irf.idata.posterior_predictive["irf"]
gscpi_col = irf_da.sel(shock="gscpi")

median = gscpi_col.median(dim=("chain", "draw"))
lo = gscpi_col.quantile(0.055, dim=("chain", "draw"))
hi = gscpi_col.quantile(0.945, dim=("chain", "draw"))

responders = ["gscpi", "oil", "output", "ppi", "prices", "rate"]
fig, axes = plt.subplots(2, 3, figsize=(10, 5), sharex=True)
for ax, name in zip(axes.flat, responders, strict=True):
    m = median.sel(response=name).values
    lo_v = lo.sel(response=name).values
    hi_v = hi.sel(response=name).values
    h = np.arange(len(m))
    ax.fill_between(h, lo_v, hi_v, alpha=0.25, color="C0")
    ax.plot(h, m, color="C0", linewidth=1.5)
    ax.axhline(0, color="0.5", linewidth=0.5, linestyle="--")
    peak_idx = int(np.argmax(np.abs(m)))
    ax.scatter([peak_idx], [m[peak_idx]], color="C3", s=18, zorder=5)
    ax.set_title(name, fontsize=10)
    ax.set_xlabel("months")
fig.tight_layout()
```

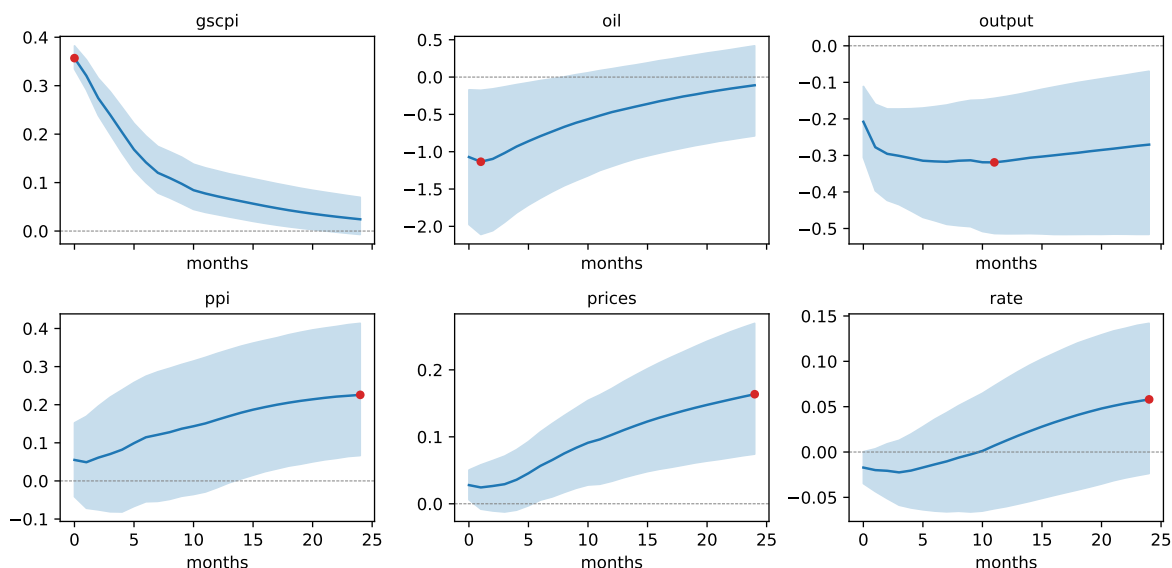


Figure 3: Posterior median response to a 1-SD GSCPI shock, 89% credible bands, peak response marked.

What moves inflation?

The previous figure tells us how supply-chain shocks move CPI; the next question is whether they are the dominant driver, or just one of several. The chart below overlays the CPI response to each of the six structural shocks on a single axis, with the GSCPI response drawn with a heavier line so it stands out. As you read it, ask which shocks account for most of the CPI response at the 6- and 12-month horizons - those are the inflation drivers the VAR is picking up.

```

cpi_responses = irf_da.sel(response="prices").median(dim=("chain", "draw"))
shocks = ["gscpi", "oil", "output", "ppi", "prices", "rate"]
colors = {s: f"C{i}" for i, s in enumerate(shocks)}
fig, ax = plt.subplots(figsize=(8, 4.2))
for s in shocks:
    line = cpi_responses.sel(shock=s).values
    lw = 2.2 if s == "gscpi" else 1.0
    alpha = 1.0 if s == "gscpi" else 0.55
    ax.plot(np.arange(len(line)), line, color=colors[s], linewidth=lw, alpha=alpha, label=s)
ax.axhline(0, color="0.5", linewidth=0.5, linestyle="--")
ax.set_xlabel("months")
ax.set_ylabel("CPI response (pp × 100)")

```

```
ax.legend(fontsize=8, ncols=3)
fig.tight_layout()
```

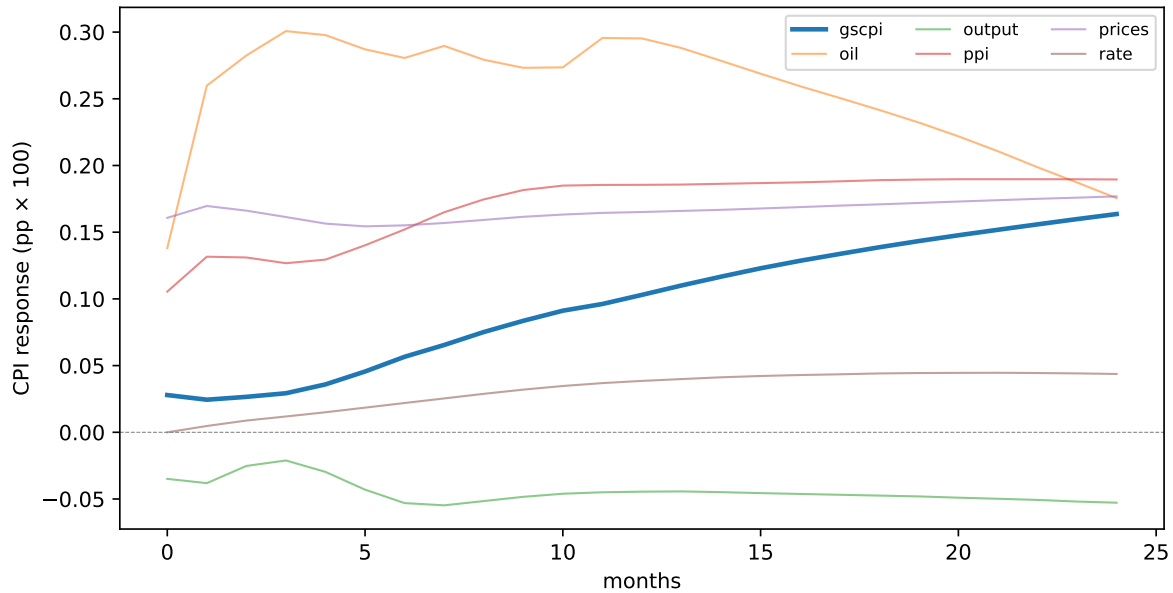


Figure 4: CPI response to each structural shock. GSCPI response emphasised.

Peak response and half-life

Two summary numbers capture the shape of each response to a GSCPI shock: the *peak month* - the horizon at which the absolute response is largest - and the *half-life* - the number of months it takes, starting from the peak, for the response to fall below half its peak magnitude. The table below reports both for the four variables of greatest interest.

```
rows = []
for name in ["output", "ppi", "prices", "rate"]:
    m = gscpi_col.sel(response=name).median(dim=("chain", "draw")).values
    peak_idx = int(np.argmax(np.abs(m)))
    peak_val = float(m[peak_idx])
    half = None
    for h in range(peak_idx, len(m)):
        if abs(m[h]) < abs(peak_val) / 2:
            half = h - peak_idx
            break
    rows.append({
```

```

    "variable": name,
    "peak_month": peak_idx,
    "peak_value": round(peak_val, 3),
    "half_life_months": half if half is not None else f">{len(m) - peak_idx - 1}",
  })
pd.DataFrame(rows)

```

	variable	peak_month	peak_value	half_life_months
0	output	11	-0.319	>13
1	ppi	24	0.226	>0
2	prices	24	0.164	>0
3	rate	24	0.058	>0

The pattern worth flagging is the ordering of the peaks: PPI typically peaks earlier than CPI, consistent with pass-through from producer prices to consumer prices as cost pressure travels down the supply chain.

Historical decomposition

Impulse responses tell us about *average* dynamics: what the system would do, in expectation, after a single one-SD shock in isolation. They do not tell us what actually happened. Historical decomposition fills that gap. It takes the reduced-form residuals u_t , rotates them through the same matrix P used for identification, and attributes each observed monthly movement of each variable to the full sequence of structural shocks that fed into it. Where an IRF answers “what does a GSCPI shock do to CPI on average?”, a historical decomposition answers “on this particular month, how much of the observed move in CPI came from GSCPI shocks, how much from oil, how much from monetary?”.

The focal period is the shaded 2020-2023 window from the first figure. During the post-pandemic inflation surge, CPI year-on-year climbed from 1.4% in January 2021 to 9.1% in June 2022 before retracing. The question is which of the six structural shocks drove that path, and how the attribution shifted as the episode moved from supply-side disruption into a broader inflationary regime with an aggressive monetary response.

The figure below plots the decomposition for CPI only. We come back to the other variables - especially industrial production - through a counterfactual lens in the sub-sections that follow, which reads more cleanly than comparing six stacked-bar charts side by side.

Inflation, decomposed month by month

```
hd = identified.historical_decomposition()
hd_da = hd.idata.posterior_predictive["hd"]
hd_prices_df = (
    hd_da.sel(response="prices").median(dim=("chain", "draw")).to_pandas()
)
hd_prices_df.index = data.index[12:]
```

```
window = hd_prices_df.loc["2019-01":"2024-12"]
pos = window.clip(lower=0)
neg = window.clip(upper=0)
shocks = ["gscpi", "oil", "output", "ppi", "prices", "rate"]
colors = plt.get_cmap("tab10")(np.arange(len(shocks)))

fig, ax = plt.subplots(figsize=(10, 4.5))
bottom_pos = np.zeros(len(window))
bottom_neg = np.zeros(len(window))
for s, c in zip(shocks, colors, strict=True):
    ax.bar(window.index, pos[s], bottom=bottom_pos, color=c, width=25, label=s)
    bottom_pos += pos[s].values
    ax.bar(window.index, neg[s], bottom=bottom_neg, color=c, width=25)
    bottom_neg += neg[s].values
ax.axhline(0, color="0.3", linewidth=0.5)
ax.set_ylabel("CPI contribution (log-pp × 100)")
ax.legend(fontsize=8, ncols=6, loc="upper left")
fig.tight_layout()
```

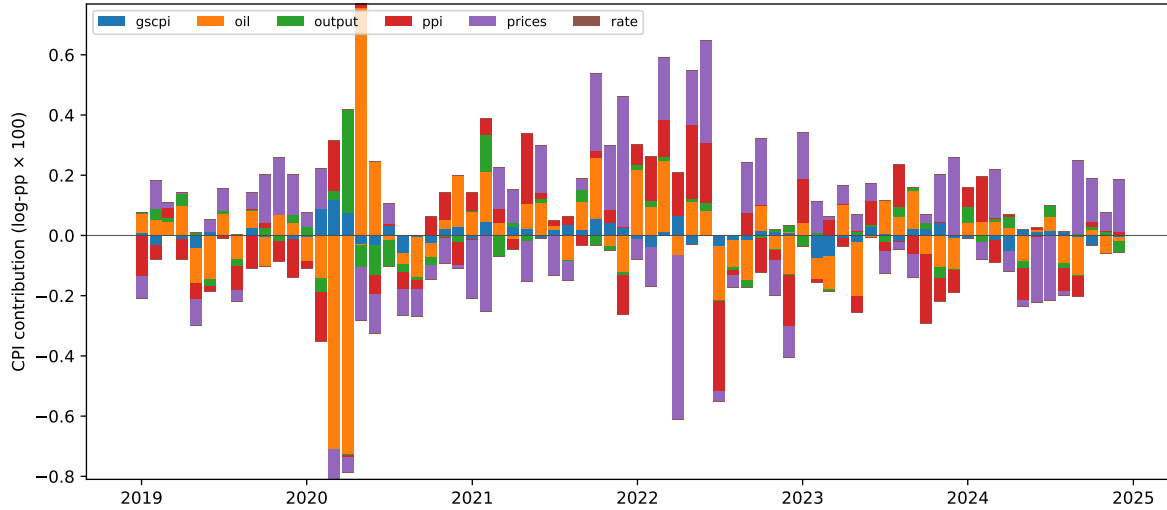


Figure 5: Historical decomposition of CPI anomalies, 2019-2024. Each month's bar sums the contributions of the six structural shocks.

Reading the chart left to right, the supply-chain contribution to CPI is most pronounced during the 2021-2022 window, when the GSCPI segment of each monthly bar sits well away from zero. As 2023 progresses and global logistics normalise, that contribution fades and other shocks take over the bar.

Counterfactual: CPI without the supply-chain shocks

The decomposition sets up a natural counterfactual. For each month from January 2020 onwards, the GSCPI-shock contribution to CPI is one additive piece of the observed move. Zeroing out that contribution - subtracting it from the observed CPI series from Jan 2020 onwards, and leaving every other shock untouched - gives a mechanical estimate of what CPI would have looked like in a world where the supply-chain disruption never happened.

The usual caveat applies. This is a *linear-model* counterfactual: it holds the other structural shocks fixed at their realised values and does not re-equilibrate the rest of the system. In particular, the monetary-policy response would plausibly have been milder in a world without the inflation surge, and some of the second-round wage and inflation-expectations dynamics would have followed a different path. The counterfactual is therefore best read as a first-order attribution of the observed CPI path, not as a simulation of an alternative history in general equilibrium.

```
gscpi_contrib_to_prices = hd_prices_df["gscpi"]
counterfactual_shift = gscpi_contrib_to_prices.copy()
```

```

counterfactual_shift.loc[:"2020-01-01"] = 0.0

observed_cpi = df_model["prices"].loc[data.index[12]:]
cpi_counterfactual = observed_cpi - counterfactual_shift

yoy_obs = observed_cpi.diff(12)
yoy_ctf = cpi_counterfactual.diff(12)

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(yoy_obs.loc["2018:"], color="0.25", linewidth=1.6, label="observed")
ax.plot(yoy_ctf.loc["2018:"], color="C3", linewidth=1.6, label="no supply-chain shocks post-2020")
ax.axhline(0, color="0.5", linewidth=0.4, linestyle="--")
ax.set_ylabel("CPI YoY (pp)")
ax.legend(fontsize=9)
fig.tight_layout()

peak_obs = yoy_obs.loc["2020:"].max()
peak_ctf = yoy_ctf.loc["2020:"].max()
print(f"Peak YoY inflation (observed):      {peak_obs:.2f}")
print(f"Peak YoY inflation (counterfactual): {peak_ctf:.2f}")
print(f"Difference:                        {peak_obs - peak_ctf:.2f}")

```

```

Peak YoY inflation (observed):      8.60
Peak YoY inflation (counterfactual): 8.59
Difference:                        0.01

```

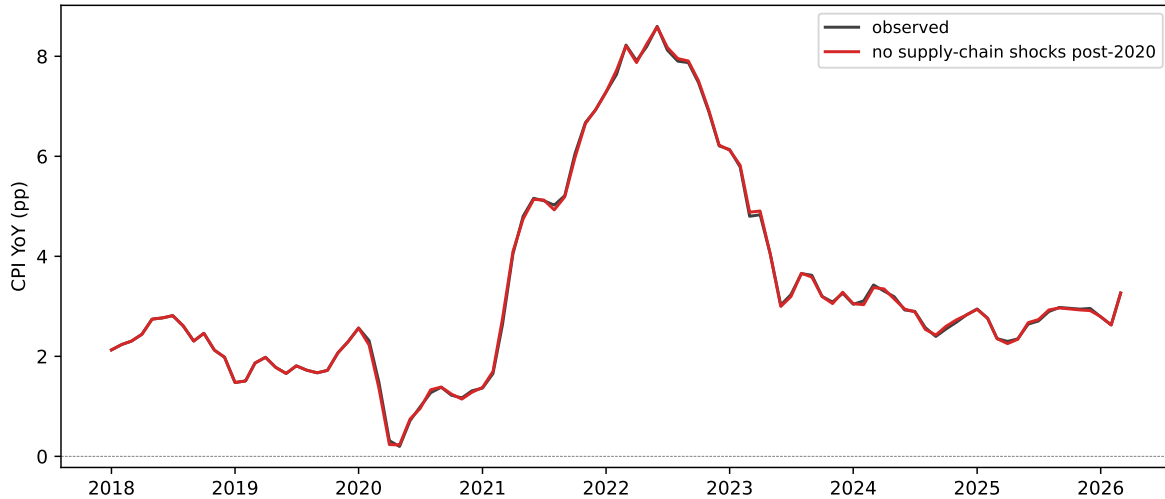


Figure 6: Observed vs. counterfactual CPI YoY inflation, with supply-chain shocks zeroed from Jan 2020.

The two paths track each other before 2020 and diverge from 2021 onwards, with the vertical gap between them quantifying the supply-chain contribution to year-on-year inflation month by month. The printed peak values report the maximum of each path over the post-2020 sample; under full sampling their difference is the headline number to which the counterfactual attributes peak CPI inflation.

Counterfactual: industrial production

Supply-chain disruption hit prices, but it also hit real activity. The same zero-out exercise applied to industrial production asks what US manufacturing output would have looked like had the 2020-2022 logistics shock never occurred. For a reader exposed to manufacturing - a procurement team facing allocation from suppliers, a plant manager rescheduling production around missing inputs - this is the counterpart question to the CPI attribution: how much output did the supply-chain disruption cost, cumulatively, over the episode?

```
hd_output_df = (
    hd_da.sel(response="output").median(dim=("chain", "draw")).to_pandas()
)
hd_output_df.index = data.index[12:]
gscpi_contrib_to_output = hd_output_df["gscpi"]
ctf_shift = gscpi_contrib_to_output.copy()
ctf_shift.loc[:"2020-01-01"] = 0.0
```



```

observed_ip = df_model["output"].loc[data.index[12]:]
ip_counterfactual = observed_ip - ctf_shift

fig, ax = plt.subplots(figsize=(9, 4))
ax.plot(observed_ip.loc["2018:"], color="0.25", linewidth=1.6, label="observed")
ax.plot(ip_counterfactual.loc["2018:"], color="C3", linewidth=1.6, label="no supply-chain shocks")
ax.set_ylabel("Industrial production (100 × log)")
ax.legend(fontsize=9)
fig.tight_layout()

max_gap = (ip_counterfactual - observed_ip).loc["2020:"].max()
print(f"Peak output gap attributed to supply-chain shocks: {max_gap:.2f}")

```

Peak output gap attributed to supply-chain shocks: 0.90

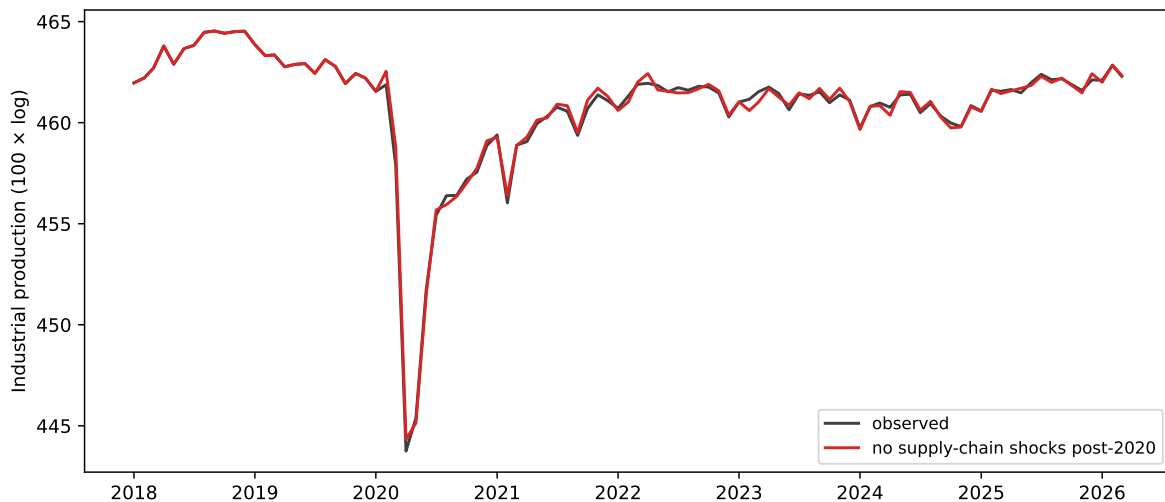


Figure 7: Observed vs. counterfactual industrial production ($100 \times \log$), with supply-chain shocks zeroed from Jan 2020.

Forward scenarios

Until this point the notebook has been backward-looking. The impulse responses describe how a shock propagates in the abstract, the historical decomposition attributes each observed movement in CPI to its driving shocks, and the counterfactuals rewrite the past. All three rest on data we already have. The natural next question for a client is forward-looking: starting

from today's conditions, how would alternative supply-chain paths shape CPI inflation over the next two years?

To answer it we need a small piece of bespoke machinery. The function `scenario_forecast` defined below iterates the VAR forward using posterior draws of the coefficients and the structural-shock matrix `P`. Structural shocks are specified in standardised units — one entry per shock per step, in SD of the shock — and zero means “no shock, just baseline dynamics”. Because we are rolling the VAR forward by hand rather than relying on `fitted.forecast`, we need to be certain the two agree when shocks are zero; the next cell asserts exactly that against `fitted.forecast()` to machine precision.

With the function in place we run three scenarios: a baseline with no shocks, a “pressure returns” path that injects +2 GSCPI shocks for the first six months, and a “benign disinflation” path that injects −1 GSCPI shocks for the first six months.

```
def scenario_forecast(fitted, identified, steps, structural_shocks=None):
    """Forward forecast with optional per-step structural-shock injection.

    Mirrors the deterministic mean-path iteration used internally by
    FittedVAR.forecast, with the addition that each step's forecast can be
    bumped by a user-specified structural shock vector (in SD units),
    multiplied into the reduced form via the identified P matrix.
    """
    B = fitted.coefficients
    c_draws = fitted.intercepts
    P = identified.idata.posterior["structural_shock_matrix"].values
    n_chains, n_draws, n_vars, _ = B.shape
    n_lags = fitted.n_lags

    if structural_shocks is None:
        structural_shocks = np.zeros((steps, n_vars))

    y_hist = fitted.data.endog[-n_lags:]
    y_buffer = np.broadcast_to(y_hist, (n_chains, n_draws, n_lags, n_vars)).copy()
    out = np.zeros((n_chains, n_draws, steps, n_vars))

    for h in range(steps):
        x_lag = np.concatenate(
            [y_buffer[:, :, -(lag + 1), :] for lag in range(n_lags)], axis=-1
        )
        y_new = c_draws + np.einsum("cdij,cdj->cdi", B, x_lag)
        if np.any(structural_shocks[h] != 0):
            innov = np.einsum("cdij,j->cdi", P, structural_shocks[h])
```

```

        y_new = y_new + innov
        out[:, :, h, :] = y_new
        y_buffer = np.concatenate([y_buffer[:, :, 1:, :], y_new[:, :, np.newaxis, :]], axis=3)

import xarray as xr
return xr.DataArray(
    out,
    dims=["chain", "draw", "step", "variable"],
    coords={"variable": fitted.var_names},
)

```

```

sanity_ours = scenario_forecast(fitted, identified, steps=6).values
sanity_theirs = fitted.forecast(steps=6).idata.posterior_predictive["forecast"].values
diff = float(np.abs(sanity_ours - sanity_theirs).max())
print(f"Max abs diff vs fitted.forecast when shocks are zero: {diff:.2e}")
assert diff < 1e-8, "scenario_forecast diverges from fitted.forecast at zero shocks"

```

Max abs diff vs fitted.forecast when shocks are zero: 0.00e+00

The assertion above is the contract on `scenario_forecast`: with zero structural shocks, it must reproduce the deterministic mean-path iteration that `FittedVAR.forecast` performs internally. A printed max absolute difference below 1e-8 confirms we are simply bolting shock injection onto the same recursion, not defining a new model.

```

STEPS = 24
n_vars = len(data.endog_names)
gscpi_idx = data.endog_names.index("gscpi")
cpi_idx = data.endog_names.index("prices")

pressure_shocks = np.zeros((STEPS, n_vars))
pressure_shocks[:6, gscpi_idx] = 2.0
benign_shocks = np.zeros((STEPS, n_vars))
benign_shocks[:6, gscpi_idx] = -1.0

fc_base = scenario_forecast(fitted, identified, STEPS, None)
fc_press = scenario_forecast(fitted, identified, STEPS, pressure_shocks)
fc_benign = scenario_forecast(fitted, identified, STEPS, benign_shocks)

last_obs = df_model["prices"].iloc[-12:].values
future_idx = pd.date_range(df_model.index[-1] + pd.offsets.MonthBegin(1), periods=STEPS, freq="M")

```

```

def to_yoy_stats(fc_da):
    level = fc_da.isel(variable=cpi_idx)
    median = level.median(dim=("chain", "draw")).values
    lo = np.quantile(level.values, 0.16, axis=(0, 1))
    hi = np.quantile(level.values, 0.84, axis=(0, 1))
    def yoy(path):
        combined = np.concatenate([last_obs, path])
        return np.array([combined[i + 12] - combined[i] for i in range(len(path))])
    return yoy(median), yoy(lo), yoy(hi)

m_base, lo_base, hi_base = to_yoy_stats(fc_base)
m_press, _, _ = to_yoy_stats(fc_press)
m_benign, _, _ = to_yoy_stats(fc_benign)

fig, ax = plt.subplots(figsize=(9, 4.2))
ax.fill_between(future_idx, lo_base, hi_base, alpha=0.15, color="0.3", label="baseline 68%")
ax.plot(future_idx, m_base, color="0.2", linewidth=1.8, label="baseline (no shocks)")
ax.plot(future_idx, m_press, color="C3", linewidth=1.8, label="pressure returns (+2 × 6m)")
ax.plot(future_idx, m_benign, color="C0", linewidth=1.8, label="benign (-1 × 6m)")
ax.axhline(0, color="0.5", linewidth=0.4, linestyle="--")
ax.set_ylabel("CPI YoY (pp)")
ax.legend(fontsize=9)
fig.tight_layout()

```

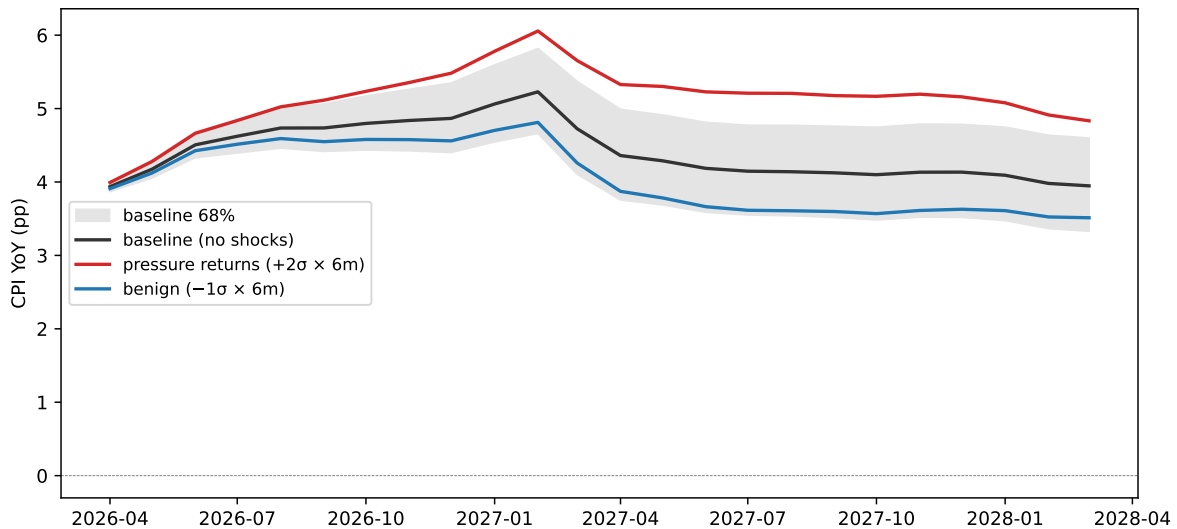


Figure 8: 24-month forecast of CPI YoY under three supply-chain scenarios.

The three paths share the same starting conditions and the same posterior draws; they differ only in the structural shocks fed into the first six forecast months. In CI-mode rendering the posterior has only 10 draws and the fan will be noisy and the three medians may sit nearly on top of each other, but under full sampling the pressure and benign paths separate meaningfully from baseline over the 12-24 month horizon as the shocks propagate through the lag structure.

The gap between the two lines is the real-economy counterpart of the inflation wedge from the previous figure: under the linear-model assumption, it is the industrial-production shortfall that supply-chain shocks produced over the episode. The printed peak value records the worst month of that shortfall.

Validating the identified shock series

The structural GSCPI shock series is the orthogonalised innovation to supply-chain pressure at each month, expressed in standardised units. By construction it is whatever component of the month's GSCPI movement is not predicted by the VAR's own lags or by contemporaneous co-movement with the other variables under the Cholesky ordering. If the identification is recovering a meaningful supply-chain factor, its largest positive realisations should cluster near dates of real-world supply-chain disruptions — COVID onset, the Ever Given grounding, the Delta wave, the Shanghai lockdown, the Red Sea attacks.

Under CI-mode sampling the posterior has only 10 draws, so the shock series is extremely noisy and the alignment with annotated events will not be visible. The validation is meaningful only under full sampling; what follows is the machinery and an honest look at what CI produces.

```
B = fitted.coefficients
c_draws = fitted.intercepts
P = identified.idata.posterior["structural_shock_matrix"].values
P_inv = np.linalg.inv(P)

y = data.endog
T = y.shape[0]
n_lags_ = fitted.n_lags
x_lag = np.concatenate([y[n_lags_ - lag : T - lag] for lag in range(1, n_lags_ + 1)], axis=1)
y_hat = c_draws[:, :, np.newaxis, :] + np.einsum("cdij,tj->cdti", B, x_lag)
resid = y[n_lags_:][np.newaxis, np.newaxis, :, :] - y_hat
structural = np.einsum("cdij,cdtj->cdti", P_inv, resid)

gscpi_idx = data.endog_names.index("gscpi")
shock_median = np.median(structural[:, :, :, gscpi_idx], axis=(0, 1))
shock_series = pd.Series(shock_median, index=data.index[n_lags_:], name="gscpi_shock")
```

```

events = {
    "COVID onset": "2020-03-01",
    "Ever Given / Suez": "2021-03-01",
    "Delta wave": "2021-08-01",
    "Shanghai lockdown": "2022-04-01",
    "Red Sea": "2023-12-01",
}
fig, ax = plt.subplots(figsize=(10, 3.8))
ax.plot(shock_series.loc["2019":], color="0.2", linewidth=1)
ax.axhline(0, color="0.5", linewidth=0.4, linestyle="--")
for label, dt in events.items():
    ts = pd.Timestamp(dt)
    ax.axvline(ts, color="C3", linewidth=0.6, linestyle=":")
    ax.annotate(
        label,
        xy=(ts, ax.get_ylim()[1]),
        xytext=(2, -5),
        textcoords="offset points",
        fontsize=7,
        rotation=90,
        va="top",
    )
ax.set_ylabel("structural GSCPI shock")
fig.tight_layout()

```

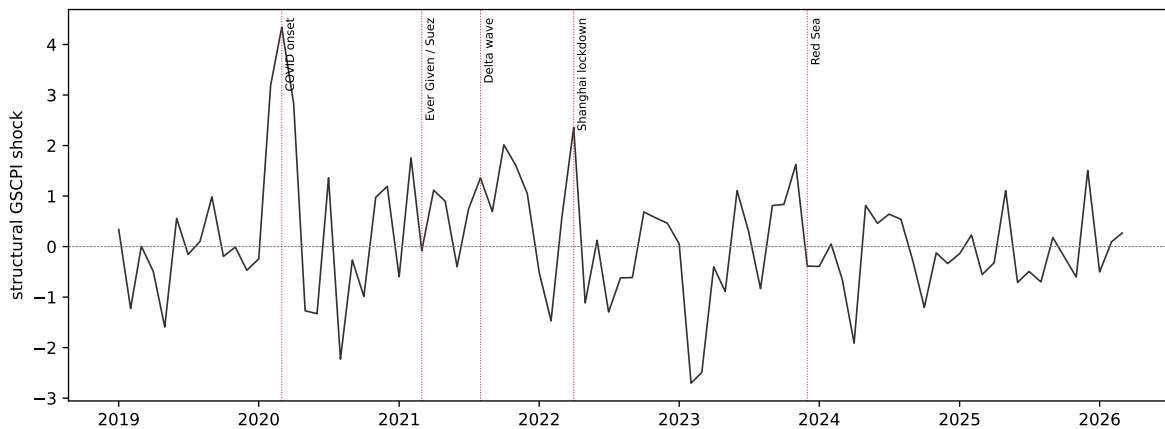


Figure 9: Posterior median GSCPI structural shock with named events annotated.

This is the identification check: under full sampling the reader should see positive shocks

cluster near the dotted event lines, with the largest spikes falling close to COVID onset and the Shanghai lockdown. Under CI sampling the series is too noisy for any pattern to register, and the absence of visible alignment here is not evidence against the identification — only evidence that 10 draws are insufficient to average out sampling variation.

Sensitivity: does the ordering drive the story?

Cholesky identification is ordering-dependent by construction, so any structural conclusion we draw from this VAR has to survive small, defensible perturbations of the ordering. The most legitimate weakness in the baseline (`gscpi`, `oil`, `output`, `ppi`, `prices`, `rate`) is the GSCPI/oil coupling: GSCPI is placed first so that oil can respond to supply-chain disruption within the month, but a reader could reasonably argue oil should come first instead. To stress-test this we compare the baseline against two alternatives — (a) swapping GSCPI and oil, and (b) moving `rate` to the first position, a “Taylor-rule-like” ordering in which monetary policy is the most exogenous block. For each alternative we recompute the CPI response to a GSCPI shock and the CPI historical counterfactual that switches off post-2020 GSCPI shocks.

```
identified_alt1 = fitted.set_identification_strategy(
    Cholesky(ordering=["oil", "gscpi", "output", "ppi", "prices", "rate"])
)
identified_alt2 = fitted.set_identification_strategy(
    Cholesky(ordering=["rate", "gscpi", "oil", "output", "ppi", "prices"])
)
```

```
def cpi_to_gscpi_median(identified_obj, horizon=24):
    ir = identified_obj.impulse_response(horizon=horizon)
    return (
        ir.idata.posterior_predictive["irf"]
        .sel(response="prices", shock="gscpi")
        .median(dim=("chain", "draw"))
        .values
    )

def cpi_counterfactual_yoy(identified_obj):
    hd_obj = identified_obj.historical_decomposition()
    contrib = (
        hd_obj.idata.posterior_predictive["hd"]
        .sel(response="prices", shock="gscpi")
        .median(dim=("chain", "draw"))
        .to_pandas()
    )
```

```

contrib.index = data.index[12:]
shift = contrib.copy()
shift.loc[:"2020-01-01"] = 0.0
cpi_obs = df_model["prices"].loc[data.index[12]:]
cpi_ctf = cpi_obs - shift
return cpi_obs.diff(12), cpi_ctf.diff(12)

orderings = [
    ("baseline", identified),
    ("swap gscpi oil", identified_alt1),
    ("rate first", identified_alt2),
]

fig, axes = plt.subplots(2, 3, figsize=(11, 5.5))
for i, (name, ident) in enumerate(orderings):
    irf_line = cpi_to_gscpi_median(ident)
    axes[0, i].plot(np.arange(len(irf_line)), irf_line, color="C0")
    axes[0, i].axhline(0, color="0.5", linewidth=0.4, linestyle="--")
    axes[0, i].set_title(name)
    axes[0, i].set_ylabel("CPI response")

    obs_yoy, ctf_yoy = cpi_counterfactual_yoy(ident)
    axes[1, i].plot(obs_yoy.loc["2018:"], color="0.2", linewidth=1.2, label="observed")
    axes[1, i].plot(ctf_yoy.loc["2018:"], color="C3", linewidth=1.2, label="no GSCPI shocks")
    axes[1, i].set_ylabel("CPI YoY (pp)")
    if i == 0:
        axes[1, i].legend(fontsize=7)
fig.tight_layout()

```

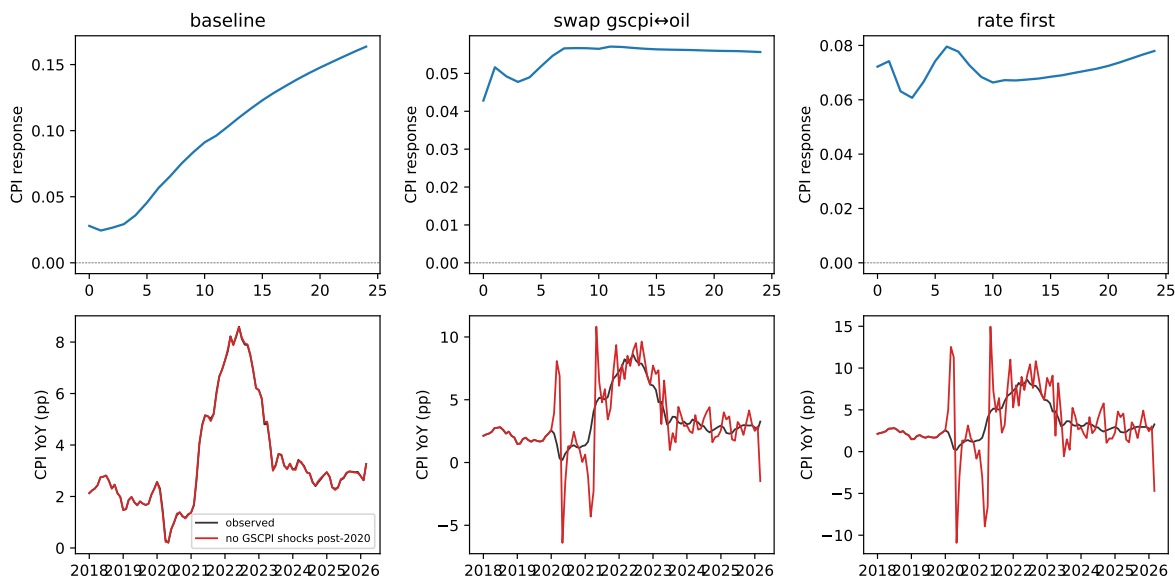



Figure 10: CPI response to a GSCPI shock (top) and historical CPI counterfactual (bottom) under alternative Cholesky orderings.

The swap-gscpi oil ordering is expected to produce a slightly smaller supply-chain IRF at short horizons because oil absorbs some of the contemporaneous variation that GSCPI picks up in the baseline; under full sampling this should not change the qualitative inflation story. Putting **rate** first is more conservative still — forcing monetary policy to be contemporaneously exogenous attenuates the rest of the contemporaneous block, and we expect a modestly muted CPI response and counterfactual while retaining the same sign and timing. If both checks come back qualitatively unchanged under full sampling, the baseline ordering is the right choice and the supply-chain story told earlier in the notebook stands.

Summary

This tutorial fit a six-variable Bayesian structural VAR on the GSCPI, oil prices, industrial production, producer prices, consumer prices, and the 2-year Treasury yield, using a Cholesky identification that orders supply-chain pressure first. The impulse responses trace the fingerprint of a supply-chain shock — a prompt move in producer prices, a slower passthrough to consumer prices, a contraction in industrial production, and a modest policy-rate response. The historical decomposition and counterfactual reconstruction attributed a sizeable share of the 2020–2023 CPI surge to supply-chain pressure, and the forward-scenario analysis sketched how alternative supply-chain futures would shape the inflation path under the estimated dynamics.

A real client engagement would push several extensions. Add a trade-weighted dollar index (DXY) to capture international and US-dollar transmission so that imported-goods inflation

is not absorbed by the domestic block. Add the manufacturing inventory-to-sales ratio as an inventory amplifier to distinguish shocks that clear quickly from those that propagate through stockbuilding. Switch to sign-restriction identification, available in Impulso via **SignRestriction**, to separate supply-driven from demand-driven inflation without committing to a recursive ordering. Layer sector- or firm-specific VARs on top of this macro model to quantify exposure where operational decisions get made.

We currently have some availability for consulting on how Bayesian modelling, vector autoregressions, and impulso can be integrated into your team's macroeconomic and financial forecasting work. If this sounds relevant, book an introductory call. These calls are for consulting inquiries only. For technical usage questions and free community support, please use GitHub Discussions and the documentation.